Republic of Yemen
Taiz University
Al-Saeed Faculty of Engineering and Information Technology

PROPERTY MANAGEMENT SYSTEM PROJECT

Prepared by Student: Mohammed Al-shameri

---

1. Project Introduction

Project Concept: The Property Management System is a dual-interface software application (offering both a Command-Line Interface in `property.management.system.py` and a modern Desktop Graphical User Interface in `pms_gui.py`) designed to manage real estate operations including property sales, purchases, and leasing. The system enforces role-based authentication and distributes permissions across four distinct user roles: **Manager**, **Employee**, **Tenant**, and **Customer**.

System Objectives:

- Manage diverse real estate inventories across three primary categories: Lands, Houses, and Apartments.
- Administer multi-role user accounts with secure login and secret-word password recovery.
- Process end-to-end real estate transactions, including sales confirmations, purchase requests, property listing offers, and lease fee calculations.
- Provide persistent data storage by automatically loading and saving records to structured external text files (`.txt`).

2. Primary Data Structures

The program utilizes Python dictionaries (`dict`) to store and manipulate records in memory during runtime before serializing them to external text files:

- **Available Property Inventories (`lands`, `houses`, `apartments`):** Nested dictionaries storing active properties with attributes such as `city`, `neighborhood`, `price`, land `size`, building `floors`, `number of apartments`, apartment `floor`, and `number of rooms`.
- **Sold Property Archives (`sold_lands`, `sold_houses`, `sold_apartments`):** Dictionaries storing properties that have been officially marked as sold and removed from active inventory.
- **User Accounts (`accounts`):** Stores credentials, profile information, security words, and role assignments for managers, employees, tenants, and customers.
- **Customer Requests (`request_of_purchase`, `request_of_sell`):** Stores incoming purchase interest requests from customers and property sale offers awaiting manager approval.

3. User Roles & Permissions

User Role	Assigned System Permissions & Capabilities
Manager	Search, view, sort, add, and edit properties; confirm property sales; manage employee accounts (view, add, edit, delete); review customer purchase requests and approve customer property sale offers.
Employee	Search, view, sort, add, and edit properties; register new tenant accounts linked to specific properties; calculate monthly and annual tenant rent payments.
Tenant	View leased property details; view rent breakdown (10% monthly rent, annual rent, 20% security deposit); auto-calculate rent across custom month durations; request contract renewal; edit personal profile data.
Customer	Search and filter available properties; submit purchase interest requests for listed properties; submit offers to list personal properties for sale; register new customer accounts.

4. Authentication & Password Recovery

Authentication is handled by the `login()` function in the CLI and the Login Card view in the GUI:

- **Credential Verification & Attempt Limiting:** Validates the entered username and password against the `accounts` dictionary, enforcing a maximum of **3 login attempts** before locking access.
- **Secret Word Recovery:** If a user forgets their password, the system prompts for their registered backup `secretword` (e.g., `CR7` or `M10`) to recover and display their password immediately.

5. Property Operations & Core Functions

Function Name	Functional Description
`property_type()`	Prompts the user to select the target property category (1: Land, 2: House, 3: Apartment, 0: Back).
`input_property_data()`	Collects common attributes (city, neighborhood, price) and category-specific attributes (size, floors, rooms).
`add_property()`	Adds a new property listing to `lands`, `houses`, or `apartments` after verifying ID uniqueness.
`edit_property()`	Allows selective modification of existing property fields (city, neighborhood, price, size/floors/rooms).
`check_property_sold()`	Confirms a property sale by transferring the record from active inventory to the corresponding `sold_*` dictionary.
`print_property_data()`	Formats and prints all key-value attributes of properties within a given dictionary.
`sort_data()`	Displays properties sorted alphabetically by city name, numerically by price, or in default order.

Function Name	Functional Description
`search_property()`	Filters properties by city name, neighborhood name, min/max price range, or full attribute specification.

6. Account & Customer Request Management

- **Employee Management (`employee_management()`):** Enables the Manager to view all employee profiles or look up a specific employee ID, add new employee accounts, update existing employee details, and delete employee accounts.
- **Customer Management (`customer_management()`):** Enables the Manager to inspect customer purchase requests (`request_of_purchase`), review customer property sale offers (`request_of_sell`), and approve sale offers so they are automatically added to the active `lands`, `houses`, or `apartments` inventory.

7. CLI Role Dashboards & User Interfaces

In the console version, `display_menu(user_role)` renders a customized menu for the authenticated user and routes execution to the corresponding controller:

- **`manager_menu()`:** Full administrative control over property inventories, sales confirmations, employees, and customer requests.
- **`employee_menu()`:** Full property management plus new tenant onboarding and tenant rent payment calculation.
- **`tenant_menu()`:** Displays the tenant's leased unit, rent breakdown, multi-month fee calculator, renewal request option, and profile editor.
- **`customer_menu()`:** Provides property search, purchase request submission, property sale offer submission, and self-registration.

8. New Feature Implementation: Graphical User Interface (Tkinter GUI)

To modernize the user experience beyond the terminal, a complete desktop Graphical User Interface was implemented in `pms_gui.py` using Python's `tkinter` and `ttk` toolkit, encapsulated in the `PropertyManagementApp(tk.Tk)` class.

8. Graphical User Interface Implementation (Tkinter GUI)

In addition to the console version, a desktop Graphical User Interface was implemented in `pms_gui.py` using Python's built-in `tkinter` and `ttk` libraries under the `PropertyManagementApp(tk.Tk)` class. Both interfaces read and write to the same text files.

GUI Component / Function	Description
Main Window & Table Styling `PropertyManagementApp.__init__()`	Initializes the main window (`1050x680`), loads all text files on startup, configures `ttk.Treeview` tables, and sets up automatic data saving when the window is closed.
Login Screen & Quick Access `show_login_screen()`	Provides username and password fields, popup dialogs for password recovery (`forgot_password_dialog()`) and customer registration (`register_customer_dialog()`), and quick demo login buttons for testing each role.
Properties Tab & Search Filter `build_properties_tab()`	Displays Lands, Houses, and Apartments in a unified `Treeview` table. Users can filter by property type, search by ID, city, or neighborhood, and sort rows by price or city.

Add, Edit & Sale Actions
`add_property_dialog()`
`edit_selected_property()`
	Displays buttons according to the user's role: adding and editing properties for Managers and Employees, confirming sales for Managers (`mark_selected_sold()`), and sending purchase or sale requests for Customers.

Employees & Requests Tabs
`build_employees_tab()`
`build_requests_tab()`
	Allows the Manager to view, add, and delete employee accounts in a table, view customer purchase requests and sale offers, and approve sale offers to move them into available properties.

Tenant Rent Calculators
`build_employee_tenant_tab()`
`build_tenant_dashboard()`
	Allows Employees to select a tenant and calculate their 10% monthly rent, annual rent, and 20% security deposit. Gives Tenants a dashboard showing their leased property, a multi-month rent calculator, and a contract renewal button.

9. User Account Structure & File Persistence

Account Schema: Each user record in `accounts` stores `name`, `email`, `phone`, `password`, `secretword`, and `role` (`manager`, `employee`, `tenant`, or `customer`), along with leased property details for tenants.

Data Persistence Functions:

- `save_to_file(file, data)` & `save_all_data()`: Serializes all in-memory dictionaries into UTF-8 encoded `.txt` files (`property_l.txt`, `property_houses.txt`, `property_apartment.txt`, `sold.property.*.txt`, `purchase_requsets.txt`, `offers_requset.txt`, and `users.account.txt`). In the GUI, data is saved automatically after every modification, upon logout, and when closing the window (`WM_DELETE_WINDOW`).
- `load_from_file(file, default_data)` & `load_all_data()`: Reads and parses the external text files at application startup, automatically falling back to default initial data if the files do not yet exist.

10. Main Application Execution Flow

1. **Initialization & Data Loading:** Loads all property, transaction, and user account dictionaries from their respective `.txt` files.
2. **Authentication Screen:** Presents the login interface (console prompt in CLI or Login Card in GUI) with password recovery and customer registration options.
3. **Role-Based Routing:** Directs the authenticated user to their designated dashboard (Manager, Employee, Tenant, or Customer) with role-appropriate tools and tabs.
4. **Safe Termination & Persistence:** Upon logging out or exiting the application, commits all updated dictionaries back to disk so no changes are lost between sessions.